

Shifting Gears: Integrating Security Audits into Automotive DevSecOps

Balasubramanian Achuthan, Mohamed Ameen Alimohideen
Automotive Security, Harman International, Bengaluru, India
{balasubramanian.achuthan, MohamedAmeen.Alimohideen}@harman.com

Abstract—The accelerated expansion of the automotive industry is concurrently experiencing a shift towards Software Defined Vehicles (SDVs) and centralized E/E architectures. This transition, while simplifying hardware, introduces new potential attack vectors and escalates the overall system complexity. This complexity multiplies as the automotive systems involve n-tier suppliers, with components delivered and integrated separately, providing attackers with fresh vulnerabilities to exploit. The high cost of rectifying vulnerabilities in production has compelled Original Equipment Manufacturers (OEMs) to adopt proactive security measures within their DevOps. The industry is now adopting "shift-left" strategies, incorporating cybersecurity tools into their CI/CD pipelines, and starting to realize that security is more of a process than a feature. Although the current DevOps toolchain addresses a significant portion of security best practices, a critical gap persists in the ability to perform an automated security audit. This paper investigates the need for the automated security audit as part of the DevOps and its and proposes a usable security methodology that Original Equipment Manufacturers (OEMs) and n-Tier suppliers can utilize to bolster their product security.

Keywords—Security Audit, DevOps Audit, Automotive Audit

I. INTRODUCTION

The automotive industry is currently undergoing a transformative phase of unparalleled magnitude. The emergence of Software Defined Vehicles (SDVs) and centralized Electronic/Electrical (E/E) architectures, while propelling advancements in vehicle design and functionality, also ushers in a formidable challenge: cybersecurity. The "Automotive Software and Electronics Report 2030" underscores a pivotal shift in the automotive industry, with a growing emphasis on software due to the convergence of Autonomous driving, connected vehicles, powertrain electrification, and shared mobility. The report highlights the expanding attack surface of connected vehicles, characterized by millions of lines of code, calls for a radical shift in security practices [1]. This increased attack surface is a product of two primary trends. First, the essence of SDVs heavily depends on software for fundamental functionalities – ranging from engine control to driver assistance systems. Second, the escalating prevalence of connected vehicle features, such as remote diagnostics and over-the-air updates and other v2x technologies paves the way for new entry points for potential attackers. As the industry evolves, there's a noticeable transition towards centralized electrical/electronic architectures, replacing the traditional independent, function-specific ECUs. However, this centralization brings with it heightened cybersecurity concerns. This paper "Centralized E/E Architecture and Evolution" talks about the cybersecurity concerns of different E/E architecture and, presenting potential vulnerabilities that could be exploited by attackers [2]. Therefore, as the industry continues to prioritize software, it must also address these emerging security challenges.

The automotive threat landscape stands out due to the potential for severe physical consequences. Unlike other devices, a hacked car could cause accidents or injuries, i.e. a security vulnerability can lead to a safety incident to a road user. This elevates the urgency of automotive cybersecurity. Furthermore, complex supply chains, long vehicle lifespans (approximately 15+ years) with outdated software, and integration with external systems create unique challenges. A vast attack surface across numerous Electronic Control Units (ECUs) further complicates matters. To address these issues, the automotive industry is adopting stricter regulations and prioritizing security throughout the development process. The repercussions of neglecting these cybersecurity risks are grave for the OEMs. A single vulnerability can trigger a catastrophic impact, potentially resulting in remote control of safety-critical systems or extensive data breaches. Also, the expense of rectifying vulnerabilities post-production significantly outweighs the cost of proactive measures implemented during development. To circumvent these expensive and potentially hazardous situations, the automotive industry requires a more proactive strategy for identifying vulnerabilities, weaknesses, and risks during the development stage. This can be realized through DevOps (Development, Security, Operations) practices, which weave security considerations throughout the entire software development lifecycle.

Though existing DevSecOps tools like SAST (Static Application Security Testing), DAST (Dynamic Application Security Testing) and Static Code Analysis provide significant security coverage, there exist few gaps and limitations [3], specifically for the Automotive domain. "Security Tools in DevSecOps – A Systematic Literature Review" highlights the limitations of the existing DevSecOps tools [4]. One of the primary concern is - Legacy Systems and Diverse File Types: Traditional tools may struggle to handle the unique file formats and programming languages used in legacy automotive systems, potentially creating security gaps [1]. Secondly, Open-Source vs. Proprietary Code Focus: While adept at analyzing open-source components, these tools often lack comprehensive coverage for proprietary code developed in-house or by vendors. Thirdly, Subjectivity in Security Concepts - Certain security principles like "Defense in Depth" or "Least Privilege" are context-dependent and require manual review as existing tools lack generic implementations. Finally, Emerging Programming Languages - The adoption of modern languages like Kotlin and Rust in automotive development necessitates new security testing approaches for these evolving technologies. While DevSecOps tools offer valuable functionalities, they cannot fully cover a comprehensive cybersecurity audit for automotive systems.

To address these challenges and ensure the security of futuristic automotive systems, novel DevSecOps approaches are required. This paper presents a comprehensive framework that integrates advanced security techniques, tailored to the

automotive domain. Our proposed framework leverages concept of introducing audits as part of the DevOps life cycle.

The remainder of this paper is organized as follows. Section II provides a detailed overview of what is an Audit in the context of cybersecurity and how its importance. Section III presents our proposed framework, highlighting its key components, such as its methodology, implementation strategy. Section IV discusses the criteria for the audit framework. Finally, Section V, VI, VII discusses the Automated audit framework and its modules like – Code review audit, Build time audit and Run time audit. This section also covers the basic algorithms that can be used for implementation of these logs.

II. WHAT AND WHY – AUDIT?

According to Wikipedia, an information security audit is “an independent review and examination of system records, activities, and related documents” [5]. This paper defines the Security audit as a verification of the system under study against a defined set of guidelines, best practices, or standards. Usually, an audit happens at the very end of the product cycle and is more of a manual process, but this paper proposes a methodology to define security audits and integrate along with other DevOps tools. This ideology is in line to the statement – “security is more of a process than a feature”. The benefits of incorporating security audits into your DevSecOps workflow are numerous. First, they offer a clear picture of your current security maturity level by comparing your practices to industry benchmarks and established best practices. This allows you to identify areas for improvement and prioritize your security efforts. Second, security audits play a vital role in building trust and fostering compliance. By demonstrating your commitment to data security and adherence to relevant regulations (e.g., ISO 26262, ISO 21434), you can build trust with stakeholders and customers. The Figure 1 demonstrates how an audit framework can help in understanding the deviation from the defined guidelines.

In this paper, our primary focus will be on an In-Vehicle Infotainment (IVI) unit that operates on the Automotive Android operating system [6]. To streamline our discussion and analysis, we will consume the Automotive android as is without any modification to demonstrate our audit test cases and framework examples.

III. METHODOLOGY

This section outlines a methodology for describing the scope and framework of security audits within software development environments. The proposed approach aims to integrate seamlessly with existing CI/CD pipelines, ensuring that security audits are both comprehensive and efficient. The methodology unfolds in three pivotal stages:

A. Scope Identification for System Audit:

To ascertain the scope the first step is to conduct a thorough analysis of the system under consideration to understand its architecture, dependencies, and external interfaces. Identify critical components and data flows that are essential for the system's operation and could potentially be vulnerable to security threats. Engage with stakeholders to understand business requirements, compliance needs, and

specific security concerns that should be addressed. The first step is a byproduct of the ISO 21434 Cybersecurity Management System [7], which is a standard cybersecurity standard dedicated for an automotive system.

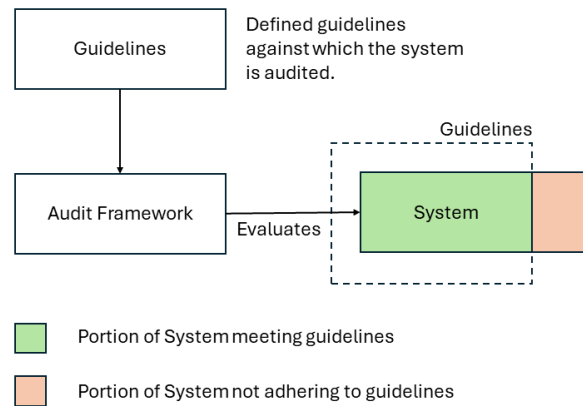


Figure 1 Depicts how an audit framework evaluates the system against the defined guidelines.

B. Scope Identification for System Audit:

To ascertain the scope the first step is to conduct a thorough analysis of the system under consideration to understand its architecture, dependencies, and external interfaces. Identify critical components and data flows that are essential for the system's operation and could potentially be vulnerable to security threats. Engage with stakeholders to understand business requirements, compliance needs, and specific security concerns that should be addressed. The first step is a byproduct of the ISO 21434 Cybersecurity Management System [7], which is a standard cybersecurity standard dedicated for an automotive system.

Listed below are some of the input sources for audit scope definitions. These input sources are key in defining the central guidelines, which acts as a reference for the audit framework. The System under study will be cross referenced against these defined guidelines.

Threat Modeling (ISO 21434): The audit scope definitions draw from several key input sources, among which Threat Modeling, as outlined in ISO 21434 for automotive systems, plays a pivotal role. This approach involves leveraging the cybersecurity goals identified during the threat analysis and risk assessment phases. By focusing on threats that are both highly severe and likely, the audit scope is meticulously aligned with the most critical security objectives. Additionally, TARA (Threat Analysis and Risk Assessment) Security controls are integral to this process, ensuring that the audit comprehensively addresses the identified risks and vulnerabilities within the automotive systems. The security controls should include audit requirements that clearly drafts the need for continuous checks throughout the development cycle.

Security Principles: Incorporating security principles into audit criteria involves translating foundational concepts such as Least Privilege [8], Defense in Depth [9], Deny-by-Default, Separation of Privilege [10], and Zero Trust security model [11] into specific, actionable audit points. For instance, to embody the principle of Least Privilege, an audit might scrutinize user access controls and privilege assignments,

ensuring that individuals only have the access necessary for their roles. Similarly, the principle of Defense in Depth could be reflected in an audit through a thorough review of firewall configurations, verifying that multiple layers of security are in place to protect against potential breaches. This approach ensures that audits are grounded in established security principles, providing a robust framework for assessing and enhancing system security.

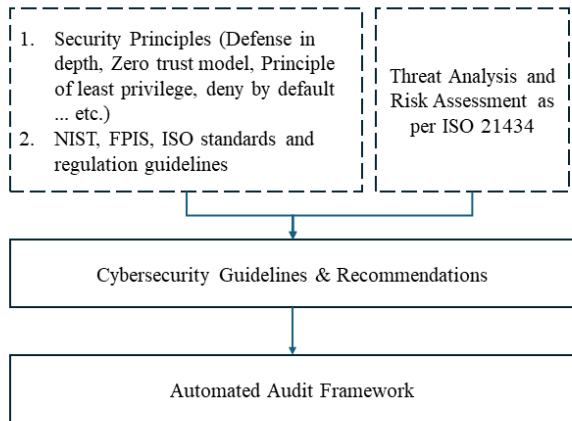


Figure 2 Different input sources for the Cybersecurity audit guidelines.

Industry Best Practices: Incorporating industry best practices into audit procedures ensures that the security measures are aligned with the highest standards. By referencing authoritative sources such as NIST, FIPS, and OWASP, audits can be structured around proven frameworks and guidelines that are widely recognized for their effectiveness in enhancing security. Additionally, considering the specific security controls recommended for Software-Defined Vehicles (SDVs) and automotive Electrical/Electronic (E/E) architectures further tailors the audit process to the unique challenges and requirements of the automotive industry. This approach not only bolsters the security posture but also ensures that the audits are comprehensive and relevant to the specific technologies and frameworks employed within the sector.

C. Audit Framework and Test Case Development:

Based on the identified scope, develop a tailored audit framework that encompasses relevant security domains such as authentication, authorization, data encryption, and code integrity. Create detailed test cases for each domain, focusing on both common vulnerabilities and those specific to the system's technology stack and architecture. Ensure that the framework and test cases are adaptable to accommodate changes in the system or emerging security threats. This paper acknowledges the fact that, not all input sources mentioned in the previous section can be converted to an automated audit framework. Considering an example of a principle of least privilege, it is practically not possible to implement an audit framework that can ascertain that the system under study is following principle of least privilege through a suite of test cases. But rather, it is important to write test cases that can support manual audit process. Some of the possible test case include – validating the process isolation, auditing access controls, sandboxing techniques ...etc.

D. Process definition and integration.

The final step is to define a usable process and integrate the security audit framework within the existing CI/CD pipeline, automating the execution of test cases as part of the development and deployment process. Configure the CI/CD tools to provide real-time feedback to developers, highlighting vulnerabilities and suggesting remediation steps. Establish mechanisms for continuous improvement of the audit process, incorporating feedback from audit results, changes in technology, and evolving security best practices.

IV. THE AUDIT FRAMEWORK CRITERIA

To ensure the effectiveness and efficiency of the audit framework, it must adhere to the following professional standards:

1. **Real-Time Inline Feedback:** The framework should integrate seamlessly into the development process, providing immediate feedback to developers. This facilitates the prompt identification and remediation of potential security issues, enhancing the overall security posture without disrupting the development workflow.
2. **Minimal Overhead:** The implementation of the audit framework should not impose significant additional burdens on developers. It must be designed to operate efficiently, ensuring that security audits do not become a bottleneck in the development process.
3. **User-Friendly Interface:** Accessibility and ease of use are paramount. The framework should be intuitive, requiring minimal training for developers to effectively utilize its features. This ensures widespread adoption and maximizes the framework's impact on improving security practices. Furthermore, in alignment with principles from the subfield of usable security, the framework must be designed with a focus on user-friendly interface design, ensuring simplicity and sophistication.

V. AUTOMATED AUDIT FRAMEWORK

The proposed audit framework comprises of 3 major modules integrated as part of the CI/CD pipeline at the different stages. The below figure illustrates proposed audit framework and its integration to a pipeline. The goals are to meet all the Audit framework criteria defined earlier making it a more usable framework.

A. Code Review Audit

The first stage involves executing a set of audit test cases during the peer-review process, immediately after a developer submits code for review. This audit is designed not to replace the invaluable step of peer review but to augment it by focusing on the security aspects of the code. The primary goal is to assist peer reviewers in evaluating the code from a security perspective, addressing one of the major challenges in multi-tier development environments: the varying levels of security knowledge among developers. Given that it's impractical to expect all developers to possess in-depth security expertise, this audit aims to provide constant feedback, thereby fostering a culture of secure coding practices over time. Tools like Code Sonar, which perform static code analysis, serve a similar purpose by offering ongoing feedback to both the reviewer and the developer about potential security guideline violations. This process not

only educates but also gradually improves the security awareness and capabilities of the development team.

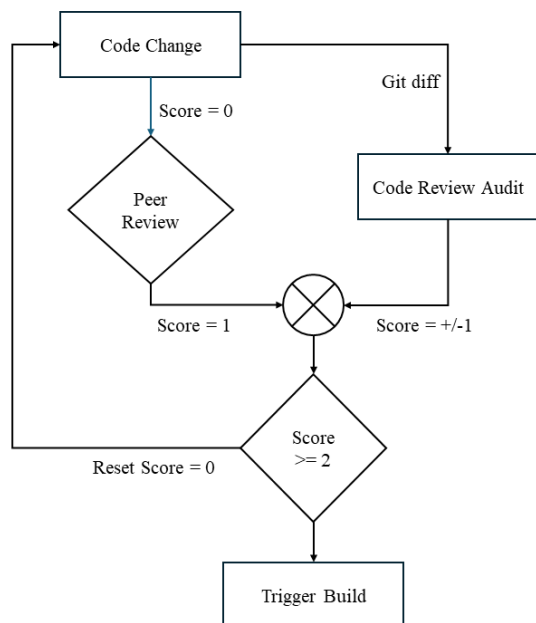


Figure 3 Code review audit process flow and scoring system.

The Figure 3 referenced delineates the integration of a code review audit process within the peer-review stage, leveraging a scoring mechanism to evaluate code submissions. In the context of Gerrit—a web-based collaboration platform for git repository management—this process is facilitated through an end-to-end flow that allows for the approval or rejection of git changes based on a simple scoring system. Although the framework can be used to function independently of such a scoring system, for explanatory purposes, we consider new code changes to initially hold a score of 0. Upon completion of the peer-review, this score is adjusted to 1. Subsequently, the differential of the modified code is forwarded to the code review audit component, which conducts a series of operations to ascertain compliance with predefined guidelines. Should the code change align with these guidelines, its score is incremented by 1; otherwise, it incurs a negative score. The cumulative score, derived from both the code review component and the peer-review, determines the eligibility of the code change to progress to the subsequent stage. A final score below 2 indicates that the code change does not meet the necessary criteria for advancement.

Implementation of Code Review component. The code review component is architected around a core parser that leverages an advanced Regex pattern matching engine, complemented by a database designed to catalog a comprehensive list of string patterns. This component is further enhanced by a suite of configurations, termed as rules, and an output formatter to streamline the review process. The efficacy of this component critically hinges on the performance of the Regex pattern matching engine. A suboptimal engine could significantly impede the review process, especially when parsing a voluminous set of patterns against a large corpus of strings.

To mitigate this potential bottleneck, this paper advocates for the adoption of Intel's Hyperscan—a state-of-the-art regular expression matching engine renowned for its high-performance capabilities[12], [13]. Hyperscan distinguishes itself by enabling the simultaneous matching of multiple

expressions, which are pre-compiled into a Hyperscan-compatible database. This database then serves as the foundation for the engine's scanning operations. The superior performance metrics of the Hyperscan engine not only fulfill but exceed the primary requirements outlined for the audit framework, ensuring an efficient and effective code review process.

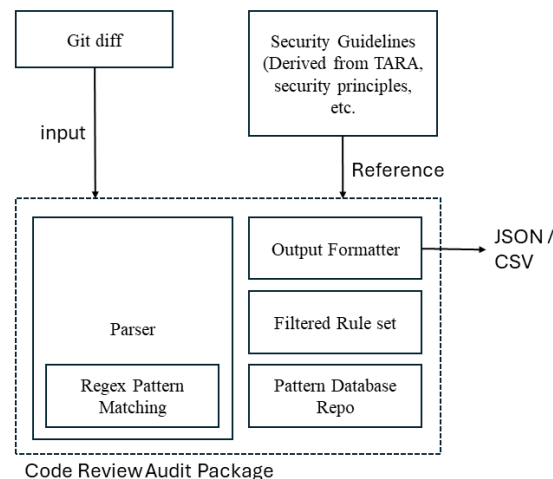


Figure 4 High level block diagram of the code review audit package.

To mitigate this potential bottleneck, this paper advocates for the adoption of Intel's Hyperscan—a state-of-the-art regular expression matching engine renowned for its high-performance capabilities[12], [13]. Hyperscan distinguishes itself by enabling the simultaneous matching of multiple expressions, which are pre-compiled into a Hyperscan-compatible database. This database then serves as the foundation for the engine's scanning operations. The superior performance metrics of the Hyperscan engine not only fulfill but exceed the primary requirements outlined for the audit framework, ensuring an efficient and effective code review process.

The pattern database, integral to our audit framework, is meticulously curated from established security guidelines. It serves as a repository of regular expression (regex) patterns, each meticulously mapped to corresponding guideline documentation, inclusive of severity details and prioritization metrics. This database is dynamically queried to extract relevant rules based on the specific file type under scrutiny. For instance, when analyzing C/C++ files, the system intelligently omits regex patterns pertinent solely to Java libraries or resource configuration (.rc) files, thereby optimizing the scanning process. This selective rule application is facilitated by the parser's built-in file type detection mechanism, which accurately identifies the file type and communicates this information to the rule engine. Subsequently, the rule engine tailors the pattern database query to the identified file type, ensuring that only relevant patterns are processed. The matched patterns are then evaluated by the regex engine, and the findings are succinctly summarized by the output formatter, which generates reports in either JSON or CSV format, catering to diverse use case requirements. This streamlined process exemplifies our commitment to efficiency and precision in enhancing software security through automated audits.

1) Algorithm

The following algorithm is used for reference implementation of the code review audit.

```
Function InitializeHyperscanDB (filetype: string)
    variable rules = filtered regex for specific filetype
    variable db = Hyperscan.compile(rules)
    return db.
```

End Function

```
Function Scan (filetype: string, code_change: [] string)
    variable violations = 0
    variable db_handle = InitializeHyperscanDB(filetype)
    variable matches = db_handle.scan(code_change,
CompileOptions)
    return matches
```

End Function

This section lists some of the possible audits at the time of code review and its associated regex patterns.

Kernel: Scan for the usage of right kernel flags, disable kernel configuration that can introduce debug features, scan for any code change that can potentially disable security features that may get unnoticed once the changes are merged.

Android Manifest files: Scan the code for inclusion of dangerous permissions, permissions that affect Personally Identifiable Information (PII), custom prohibited permissions as per the org policy.

Access Control: Scan for the changes in *.rc files or *.te files that can give excess permission than needed. Scan the code for generic se-policies with generic context like system_app, platform_app ...etc, which can give excessive permission, Scan the code changes for any services that has excessive permission based on the User and Group. There are numerous opensource tools for auditing se-linux and these can be integrated as part of this framework [14].

VI. BUILD PHASE SECURITY AUDIT

At the heart of our proposed architecture lies the critical integration of security audits during the build phase. This pivotal stage is dedicated to scrutinizing the codebase for any security vulnerabilities and ensuring adherence to established security standards, all before the code moves further along the development pipeline. The primary objectives at this juncture include auditing for compile-time flags, optimization settings, obfuscation configurations, and the activation of hardening flags, with a keen focus on preventing any debug tools from inadvertently being included in the production environment.

Key Audits During the Build Phase:

- **Compiler Options and Binary Analysis:** It's essential to meticulously scan the build files to verify that the appropriate compiler options are being utilized. This ensures that the resulting binary is devoid of debug symbols and is optimized correctly for security and performance.
- **Code Obfuscation:** For languages like Java, applying obfuscation techniques is crucial to safeguard the code. Obfuscation complicates the attacker's ability to decompile and understand the *.jar files, thereby protecting the source code from easy interpretation.

- **Debug Tools Exclusion:** Vigilance is required to ensure that make and build files do not inadvertently include any debug tools within the target binary. Such tools could potentially offer attackers deeper insights into the system, facilitating unauthorized access or exploitation.

By implementing these audits during the build phase, we aim to fortify the security of the software, ensuring that it is robust against potential threats and compliant with security standards, thereby significantly reducing the attack surface available to potential attackers.

VII. RUNTIME SECURITY AUDIT

The culmination of our security audit architecture is the Runtime Security Audit phase. This critical stage entails the execution of security audits while the application is actively running, within an environment that mirrors the intended deployment setting as closely as possible. This phase is pivotal for uncovering security vulnerabilities that emerge only during the live operation of the application, thereby ensuring a holistic security evaluation across the development lifecycle. The essence of this phase lies in its ability to dynamically test the system, revealing a broad spectrum of potential security issues in real-time. The objective here is to deploy a specialized binary within the target environment, designed to continuously monitor system operations and flag any security concerns. These runtime audit tests are meant to complement, not replace, pre-existing security test frameworks.

Key Runtime Audits Include:

- **Network Audit:** Utilizing the netstat command, this audit examines the array of services and ports currently active, cross-referencing them against expected use cases to identify any discrepancies or unauthorized activities that could signify a security breach. Scanning the network for any clear text communication to detect potential data leakage or interception vulnerabilities. This step is crucial for ensuring that sensitive information is encrypted during transmission, thereby safeguarding against eavesdropping or man-in-the-middle attacks.
- **Process and Policy Audit:** By employing the ps -AZ command, this audit assesses whether all services are operating under non-root user privileges and verifies the security policy contexts (se-policy) for any anomalies, ensuring strict adherence to security best practices.
- **Trusted Execution Environment (TEE) Verification:** This audit involves checking for the presence and operational status of the Trusted Execution Environment, a critical component for securing sensitive operations and data.

Listed below are some of the algorithms that can be used to validate the File Based Encryption in Android. This audit is necessary to ensure accidental removal or these config files. These may get unnoticed during the prolonged development timelines.

Function Validate_Android_Property

Read the system property through "getprop" binary/tool

Grep "ro.crypto.type" and "ro.crypto.state"

Ensure Type is File and State is Encrypted

End Function

Function Validate_Kernel_FBE_Flags

Read the Kernel configs using "zcat /proc/config.gz"


```
Grep "CONFIG_FS_ENCRYPTION"
Grep "CONFIG_CRYPTOAES_ARM64_CE_BLK"
Grep "CONFIG_CRYPTOAES_ARM64_CE"
Ensure all the above configs are enabled
```

End Function

Function Validate_file_encryption_flag_in_fstab

Locate the active fstab file inside "/vendor/etc/"

Read the "userdata" partition entry in fstab

Grep for "file_encryption" flag and ensure the flag is present

End Function

Through these targeted runtime audits, we aim to provide a comprehensive and continuous security oversight, ensuring that the application remains secure throughout its operational lifecycle. This approach not only identifies but also helps in promptly mitigating security risks, thereby safeguarding the integrity and reliability of the system in its live environment.

By embedding these audits at strategic points in the development process, the proposed architecture aims to create a robust framework for identifying and addressing security issues, thereby enhancing the overall security of the software being developed.

VIII. CONCLUSION

The ever-increasing reliance on software within modern vehicles necessitates a paradigm shift towards proactive security measures throughout the development lifecycle. This paper proposes a comprehensive methodology for defining and implementing security audits within the automotive DevSecOps pipeline. Our approach prioritizes meticulous scoping to ensure audits are targeted and address the most critical security risks specific to each system under study. The seamless integration with the CI/CD pipeline enables continuous monitoring and early detection of vulnerabilities, fostering a "shift left" security strategy. The proposed methodology offers several advantages for strengthening security within the automotive software development landscape. By focusing on targeted audits, continuous monitoring, and developer enablement, the overall security posture of automotive software is significantly enhanced. This proactive approach contributes to the safety and reliability of modern vehicles, ultimately leading to a safer driving experience.

REFERENCES

- [1] "Automotive software electronics market 2030 | McKinsey." Accessed: Oct. 24, 2024. [Online]. Available: <https://www.mckinsey.com/industries/automotive-and-assembly/our-insights/mapping-the-automotive-software-and-electronics-landscape-through-2030>
- [2] N. V. Vignesh, M. Kumar, B. Achuthan, S. Badade, P. Shivakumar, and A. Keshri, "Centralized E/E Architecture and Evolution," in *SAE Technical Papers*, SAE International, Sep. 2023. doi: 10.4271/2023-01-1511.
- [3] S. Tatineni, "COMPLIANCE AND AUDIT CHALLENGES IN DEVOPS: A SECURITY PERSPECTIVE", doi: 10.56726/IRJMETS45309.
- [4] J. Martelleur and A. Hamza, "Security Tools in DevSecOps: A Systematic Literature Review," 2022, Accessed: Oct. 24, 2024. [Online]. Available: <https://www.diva-portal.org/smash/record.jsf?pid=diva2:1727554>
- [5] "Information security audit - Wikipedia." Accessed: Jun. 25, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Information_security_audit
- [6] "What is Android Automotive? | Android Open Source Project." Accessed: Jun. 25, 2024. [Online]. Available: https://source.android.com/docs/automotive/start/what_at_automotive
- [7] "ISO/SAE 21434:2021 - Road vehicles — Cybersecurity engineering." Accessed: Jun. 27, 2024. [Online]. Available: <https://www.iso.org/standard/70918.html>
- [8] "Principle of least privilege - Wikipedia." Accessed: Jun. 27, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Principle_of_least_privilege
- [9] "Defense in depth (computing) - Wikipedia." Accessed: Jun. 27, 2024. [Online]. Available: [https://en.wikipedia.org/wiki/Defense_in_depth_\(computing\)](https://en.wikipedia.org/wiki/Defense_in_depth_(computing))
- [10] "Privilege separation - Wikipedia." Accessed: Jun. 27, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Privilege_separation
- [11] "Zero trust security model - Wikipedia." Accessed: Jun. 27, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Zero_trust_security_model
- [12] "Introduction to Hyperscan." Accessed: Jun. 25, 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-hyperscan.html>
- [13] "Hyperscan: Performance Analysis of Hyperscan with hsbench." Accessed: Jun. 27, 2024. [Online]. Available: <https://www.intel.com/content/www/us/en/collections/libraries/hyperscan/performance-analysis-hyperscan-hsbench.html>
- [14] A. Eaman, B. Sistany, and A. Felty, "Review of Existing Analysis Tools for SELinux Security Policies: Challenges and a Proposed Solution", doi: 10.1007/978-3-319-59041-7.